2 Conditional Flow

By the end of this section, you should be able to:

- Use the conditional `if-else` statements to return an output based on a logical condition of a scalar
- Use the `if_else()` function within a `mutate()` to conduct a logical test across a single variable
- Use the `case_when()` function to implement a vectorised version of multiple `if_else()` statements

2.1 The `if` and `else` structure

An `if-else` statement is a great tool for the developer trying to return an output based on a condition.

The `if` statement takes a condition, if the condition evaluates to `TRUE`, the R code associated with the if statement is executed. The **condition** to check appears inside **parentheses** `()` and the **R code that has to be executed** if the condition is true, follows in **curly brackets** `{}`.

Even though curly brackets are not always necessary we encourage their use for consistency. According to Hadley Wickham's style guide an open curly bracket should be located at the end of a line and a closed curly bracket should always go on its own line, unless followed by `else`.

In the following example, if x is less than zero, then R prints `x is negative`. Note that since there is no `else` statement following the `if`, no code will be executed if the condition is not satisfied.

```
# Set x equal to -3
x <- -3
# If x is less than zero, execute the print command
if (x < 0) {
  print("x is negative")
}
```

```
[1] "x is negative"
```

To combine logical expressions in the condition we can use the logical operators `!, &, |` as well as functions such as `all`, `any` and `xor`. XOR is a binary operation which stands for "exclusive or" which evaluates to true only if one of the arguments parsed is true.

	x	y	xor
1	false	false	false
2	false	true	true
3	true	false	true
4	true	true	false

```
x <- 3
y <- 5
if (x != y) {
  print("x and y are not equal to each other")
}
```

```
[1] "x and y are not equal to each other"
```

```
x <- TRUE
y <- TRUE
if (all(x, y)) {
  print("x and y are both TRUE")
}
```

```
[1] "x and y are both TRUE"
```

The `else` statement doesn't need an explicit condition. Instead it has to be used together with an `if` statement. The code associated with an `else` statement gets executed whenever the condition of the `if` is not satisfied.

```
# Set x equal to 5
x <- 5
# If x is less than zero, print x is negative, otherwise print x is zero or positive
if (x < 0) {
  print("x is negative")
} else {
  print("x is zero or positive")
}
```

```
[1] "x is zero or positive"
```

2.1.1 The `else if` statement

We can further customize the control level with the `else if` statement. The `else if` statement comes in between the `if` and `else` statement and it needs an explicit condition. With `else if` we can add as many conditions as we want.

```
x <- 0
if (x < 0) {
  print("x is negative")
} else if (x == 0) {
  print("x is zero")
} else {
  print("x is positive")
}
```

```
[1] "x is zero"
```

Remember: As soon as R stumbles upon a condition that evaluates to `TRUE`, R executes the corresponding code and then ignores the rest of the control structure. This becomes important if the conditions you list are not mutually exclusive.

The condition of an `if` statement **cannot** be a vector (of length >1). If we want to carry out a conditional operation on a vector, we need to either subset it, use loops or the `if_else` function (vectorization).

2.2 The `if_else()` function

The `if_else()` function from the `dplyr` package is an alternative and shorthand form of the R if-else statement. It uses the **vectorised technique**, which makes the operation faster. A lot of functions in R are **vectorized**, meaning that the function will operate on all elements of a vector without needing to loop through and act on each element one at a time. This makes writing code **more concise, easy to read, and less error prone**. The syntax of the `if_else()` function is the following:

```
if_else(logical condition, true, false, missing=NULL)
```

Here is a simple example of how you would use `if_else()` to categorize a set of numbers as either less than or greater or equal to 5:

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
x <- c(2, 4, 6, 8, 10)
if_else(x >= 5, "greater or equal to 5", "less than 5")
```

```
[1] "less than 5"           "less than 5"           "greater or equal to 5"
[4] "greater or equal to 5" "greater or equal to 5"
```

The `if_else()` function becomes very powerful when combined with the `mutate()` function. The combo allows users to conduct a logical test across a single variable and then populate the fields of a new variable depending on the outcome of the tests.

In the following example, `if_else()` is called within `mutate()` to create a new variable called **AGE_GROUP**, that splits the subjects into two groups depending on whether or not the subjects are less than 50 years old.

```r
# Load the dplyr package
library(dplyr)

# Read in adsl
adsl <- read_sas("./data/adsl.sas7bdat")

# Create a new variable called AGE_GROUP
# If AGE is less than 50 populate AGE_GROUP as <50, otherwise populate it as >=50
adsl_age_group <- adsl %>%
  mutate(AGE_GROUP = if_else(AGE < 50, "<50", ">=50")) %>%
  select(USUBJID, AGE, AGE_GROUP)
```

Show 5 entries Search:

	USUBJID	AGE	AGE_GROUP
1	987650.000001	35	<50
2	987650.000002	62	>=50
3	987650.000003	27	<50
4	987650.000004	42	<50
5	987650.000005	59	>=50

Showing 1 to 5 of 24 entries Previous 1 2 3 4 5 Next

2.2.1 dplyr `if_else()` vs base R `ifelse()`

The `if_else()` function is more strict than base `ifelse()` function. It checks that both alternatives are of the same type and otherwise throws an error, while `ifelse()` will promote types as necessary. Let's look at an example to help explain this.

```
ifelse(c(TRUE, TRUE, FALSE), "a", 3)
```

```
[1] "a" "a" "3"
```

```
# if_else(c(TRUE, TRUE, FALSE), "a", 3)
# Error: `false` must be a character vector, not a double vector.
```

In this example we can see that when we use the **`dplyr`** `if_else()` it checks that the true and false statements are the same type and outputs an error if this is not true. Whereas, when we use the **`base R`** `ifelse()` it automatically converts the false statement to type character.

The strictness of the dplyr `if_else()` function makes the output type more predictable, and makes it somewhat faster.

Another advantage of `if_else()` is that you can attribute a value in case of NA, which is a handy way of adding an extra condition.

```
# Create some missing values for the AGE variable
# To do that we can use the function na_if to replace any AGE = 27 with NA
adsl_withna <- adsl %>%
  mutate(AGE = na_if(AGE, 27))

# If AGE is missing then set AGE_GROUP to UNKNOWN
adsl_age_group_na <- adsl_withna %>%
  mutate(AGE_GROUP = if_else(AGE < 50, "<50", ">=50", missing = "UNKNOWN")) %>%
  select(USUBJID, AGE, AGE_GROUP)
```

Show 5 entries Search:

	USUBJID	AGE	AGE_GROUP
1	987650.000001	35	<50
2	987650.000002	62	>=50
3	987650.000003		UNKNOWN
4	987650.000004	42	<50
5	987650.000005	59	>=50

Showing 1 to 5 of 24 entries Previous 1 2 3 4 5 Next

2.3 The `case_when()` function

The `case_when()` function implements a vectorised version of multiple `if_else` statements.

```
# Load dplyr
library(dplyr)

# Create a vector from 1 to 20
x <- 1:20

# Use case_when to categorize the variables into 3 groups
case_when(
  x < 5 ~ "Less than 5",
  x >= 5 & x <= 10 ~ "Between 5 - 10",
  x > 10 ~ "More than 10"
)
```

```
 [1] "Less than 5"    "Less than 5"    "Less than 5"    "Less than 5"
 [5] "Between 5 - 10" "Between 5 - 10" "Between 5 - 10" "Between 5 - 10"
 [9] "Between 5 - 10" "Between 5 - 10" "More than 10"   "More than 10"
[13] "More than 10"   "More than 10"   "More than 10"   "More than 10"
[17] "More than 10"   "More than 10"   "More than 10"   "More than 10"
```

Imagine we want to create a new variable in `adsl` called `WEIGHT_GROUP`, that would split the subjects into three groups depending on the **WEIGHTBL** variable: those less than 60, those between 60-80 and those greater than 80. To achieve this using the `if_else()` function we would either need to create multiple flags or nest `if_else()` statements. The `case_when()` syntax requires a lot less "`()`"!

```
# Create a new variable called WEIGHT_GROUP
# With nested if_else
adsl_weight_group <- adsl %>%
  mutate(WEIGHT_GROUP = if_else(WEIGHTBL < 60, "Less than 60",
    if_else(WEIGHTBL >= 60 & WEIGHTBL <= 80, "Between 60 and 80", "Greater than 80")
  )) %>%
  select(USUBJID, WEIGHTBL, WEIGHT_GROUP)


# With case_when
adsl_weight_group_2 <- adsl %>%
  mutate(WEIGHT_GROUP = case_when(
    WEIGHTBL < 60 ~ "Less than 60",
    WEIGHTBL >= 60 & WEIGHTBL <= 80 ~ "Between 60 and 80",
    TRUE ~ "Greater than 80"
  )) %>%
  select(USUBJID, WEIGHTBL, WEIGHT_GROUP)
```

Show 5 entries Search:

	USUBJID	WEIGHTBL	WEIGHT_GROUP
1	987650.000001	88	Greater than 80
2	987650.000002	82	Greater than 80
3	987650.000003	56	Less than 60
4	987650.000004	72	Between 60 and 80
5	987650.000005	60	Between 60 and 80

Showing 1 to 5 of 24 entries Previous 1 2 3 4 5 Next

The following example was taken from the ADaM R Programming Wiki. Here, we use the SDTM `lbch` dataset to derive variables `PARAMN`, `PARAM`, `PARAMCD`, `PARCAT1`, `AVAL` and `AVALC`. Notice how the `case_when()` function allows us to easily map each `LBTESTCD` to the correct `PARAMN` in a very tidy manner!

```
library(dplyr)
library(haven)


# Read in the lbch data
lbch <- read_xpt("data/lbch.xpt")

# Derive variables PARAMN, PARAM, PARAMCD, PARCAT1, AVAL, AVALC
lb_param <- mutate(lbch,
  PARAMN = as.numeric(case_when(
    LBTESTCD == "SODIUM" ~ 18,
    LBTESTCD == "K" ~ 19,
    LBTESTCD == "CL" ~ 20,
    LBTESTCD == "BILI" ~ 21,
    LBTESTCD == "ALP" ~ 22,
    LBTESTCD == "GGT" ~ 23,
    LBTESTCD == "ALT" ~ 24,
    LBTESTCD == "AST" ~ 25,
    LBTESTCD == "BUN" ~ 26,
    LBTESTCD == "CREAT" ~ 27,
    LBTESTCD == "URATE" ~ 28,
    LBTESTCD == "PHOS" ~ 29,
    LBTESTCD == "CA" ~ 30,
    LBTESTCD == "GLUC" ~ 31,
    LBTESTCD == "PROT" ~ 32,
    LBTESTCD == "ALB" ~ 33,
    LBTESTCD == "CHOL" ~ 34,
    LBTESTCD == "CK" ~ 35
```

```
  )),
  PARAM = paste(LBTEST, " (", LBSTRESU, ")", sep = "", collapse = NULL),
  PARAMCD = case_when(PARAMN < 100 ~ LBTESTCD),
  PARCAT1 = "CHEM",
  AVAL = LBSTRESN,
  AVALC = LBSTRESC
)

lb_param_head <- head(lb_param, 20) %>%
  select(USUBJID, LBTESTCD, PARAMN, LBTEST, LBSTRESU, PARAM, PARAMCD, AVAL, AVALC)
```

The dataset below shows the first 20 rows of the output:

Show 5 entries Search:

	USUBJID	LBTESTCD	PARAMN	LBTEST	LBSTRESU	PARAM	P
1	01-701-1015	ALB	33	Albumin	g/L	Albumin (g/L)	ALB
2	01-701-1015	ALB	33	Albumin	g/L	Albumin (g/L)	ALB
3	01-701-1015	ALB	33	Albumin	g/L	Albumin (g/L)	ALB
4	01-701-1015	ALB	33	Albumin	g/L	Albumin (g/L)	ALB
5	01-701-1015	ALB	33	Albumin	g/L	Albumin (g/L)	ALB

Showing 1 to 5 of 20 entries Previous 1 2 3 4 Next